Index

Name RAGAVI N [CN observation]

Std CSE (407) Sec Roll No.

School

Subject

No	Date	Title	Marks	Signature
01.	23/07/26	Study of various network commands used in Linux and Windows		
02.	30/07/26	Study of network cables		
03.	30/07/26	Experiments on CISCO PACKET TRACER (simulation tool)		
04.	03/08/26	Setting up a local area network (LAN)		
05.	10/08/26	Experiments on packet capture tool: Wireshark		
06.	10/09/26	Hamming code		
07.	17/09/26	sliding window protocol		
08.	28/09/26	(a) Nmap to discover live hosts using Nmap scans on TRYHACKME		
	28/09/26	(b) Configuration of WLAN using cisco packet tracer.		
09.	28/09/26	Implementation of subnetting using the cisco packet tracer.		
10.	28/09/26	(a) Internetworking with routers		
	28/09/26	(b) Internetwork using wireless router, DHCP & Internet cloud.		
		(c) Internetwork using		

No	Date	Title	sign.
		switch, router & Ethernet.	
11.	29/09/26	(a) Static routing configuration using cisco packet tracer	
	29/09/26	(b) RIP using cisco packet tracer	
12.	08/10/26	Echo - client - server using TCP/UDP sockets.	
13.	08/10/26	Ping program	
14.	08/10/26	Packets ~~to~~ sniffing using raw sockets.	

Completed

EXP.NO:01 STUDY OF VARIOUS NETWORK COMMANDS
28/07/2026 USED IN LINUX AND WINDOWS

AIM:

Study of various network commands used in linux and windows.

Basic networking commands:

1) arp-a: ARP is short form of address resolution protocol, it will show the IP address of your computer along with the IP address and MAC address of your router.

Output:

Interface 172.168.56.1---0x14

Internet address	Physical address	Type
192.168.66.255	ff-ff-ff-ff-ff-ff	static
224.0.0.22	01-00-5e-00-00-16	static
224.0.0.169	01-00-5e-00-00-a7	static
224.0.0.251	01-00-5e-00-00-fb	static

Interface 10.249.65.165---0x8

Internet address	Physical address	Type
10.249.65.181	52-04-50-cc-2c-26	
10.249.65.255	ff-ff-ff-ff-ff-ff	
244.0.0.22	01-00-5e-00-00-16	
224.0.0.251	01-00-5e-00-00-fb	

2) Hostname: This is the simplest of all TCP/IP commands. It displays the name of your computer.

Output:

DESKTOP-018DCPN

3) ipconfig /all: This command displays detailed configuration information about your TCP/IP connection including router, gateway, DNS, DHCP, and type of ethernet adapter in your system.

Output:

Windows IP configuration

Hostname ..... : DESKTOP-OIG OCPN

Primary DNS suffix ..... :

Node Type ..... : Hybrid

IP Routing Enabled ..... : No

WINS proxy enabled ..... : No

4) nbstat -a: This command helps solve problem with NetBIOS name resolution (Nbt stands for NetBIOS over TCP/IP)

Output:

NBTSTAT [[-a RemoteName] [A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]]

Displays protocol statistics and current TCP/IP connections using NBT

-a (adapter status) - List the remote machine name table given its name

-A (Adapter status) - List the remote machines names table given its IP address.

-c (cache) - Lists NBT's cache of remote names and their IP address.

-n(names) - Lists local NetBIOS names.

5) netstat: Displays active network connections routing table and network statics.

Proto	Local address	Foreign address	state
TCP	127.0.0.1:49678	hp-2ag-49679	Established
TCP	127.0.0.1:49679	hp-2ag-49678	
TCP	127.0.0.1:49680	hp-2ag-49671	
TCP	127.0.0.1:49681	hp-2ag-49681	
TCP	127.0.0.1:49682	hp-2ag-49680	

6) ns lookup: Used to find DNS information that is IP address of a domain and DNS records.

output:

Default server: Unknown

Address: 172.16.81

7) pathping: It is unique to window's and is basically a combination of the ping and tracert commands:

Output:

usage: Pathping [-g-host-list] [-n maximum_hops] [-i address] [-n] [-p period] [-q.num_queries] [-w timeout] [-4] [-6] target-name.

options:

-ghost_list : Loose source route along host_list

- h maximum_hops: maximum number of hops to search for target.

8) ping: (packet internet groper) command is the best way to test connectivity between two nodes.

1. #ping hostname (ping localhost)
2. #ping ip address (ping 4.2.2.2)
3. #ping fully qualified domain name (ping www.facebook.com).

output:

usage: ping [-t][-a][-n count][-l size][-f][-i TTL]
[-v Tos][-r count][-s count][[-j host_list]|
[-k host_list]][-w timeout][-R][-s
Srcaddr][-c compartment][-p][-4][-6]
target_name.

options:

-t : ping the specified host name until stopped

-a : Resolve address to hostname.

9) Route : Used to show the IP routing table. It is primarily used to setup static routes via an interface.

Output:

Manipulate network round table.

ROUTE [-f][-p] [-4|-6] command [destination]
[MASK netmask][gateway][METRIC metric]
[IF interface]

-f : clears the routing tables for all gateway entries.

10) ip address show : Displays all network interfaces and their assigned IP address.

output : enp0s3 UP

11) ip address add
192.168.1.254/24 dev enp0s03

Assigns the IP address 192.168.1.254 to interface enp0s03.

12) ifconfig : The ifconfig command was/is a staple in many sysadmin's tool belt for configu-ring and troubleshooting networks. It has since

been replaced by the ip command discussed above.

output:

eth0: flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 150 inet 192.168.1.190. net mask 255.255.255.0 broadcast 192.168.1.255 inet6 te80::a00:27ff:fe4e:66a1 prefixlen 64 scopeid 0020<link>, ether 08:00:27:4e:66: u1 txqueuelen 1000 (Ethernet).

13) mtr: MTR is a progress with command line interface that serves as a network diagnostic and troubleshooting tool.

output: My traceroute [v0:87]

Host Packets

1:::1 Loss%. Snt last avg best wist st dev

0.0% 145 0.2 0.1 0.1 0.0 0 0.0

14) tcpdum: The tcp dump command is designed for capturing and displaying packet.

output: you don't have permission to capture on that device.

Result:

Thus, the basic networking as Linux commands are learned and the output gets verified.

EXP. NO: 02

30/07/2026

STUDY OF NETWORK CABLES

AIM:

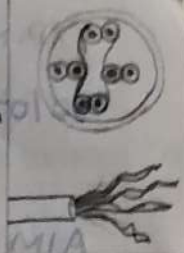
Study of different types of Network cables.

a) Understand different types of network cable.

Different types of cables used in networking are:

1. Unshielded Twisted pair (UTP) cable.
2. Shielded Twisted pair (STP) cable
3. Coaxial cable.
4. Fiber optic cable.

Cable type	Category	Maximum data transmission	Advantages/Disadvantages	Application/Use	Image
UTP	Category 3	10bps	Advantages: • cheaper in cost • Easy to install as they have a small overall diameter. Disadvantages • More prone to (EMI) and noise	10 Base-T Ethernet	
	Category 5	Up to 100 Mbps		Fast ethernet, Gigabit ethernet	
	Category 5e	1Gbps		Fast ethernet, Gigabit Ethernet	
STP	category 6, 6a	10Gbps	Advantages: • shielded • Faster than UTP. • Less susceptible to noise and interference	Gigabit ether-net, 10G Ethernet (55m) Widely used in data centres.	

SSTP	Category 7	10Gbps	Disadvantages: • Expensive • Greater installation effort.	Gigabit Ethernet, 10G Ethern-et (100m)	
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	• High bandwid-th. • Immune to interference. • Low loss bandwidth • Versatile Disadvantages: • Limited distance • Cost • size is bulky.	speed of signal is 500m Television network High speed internet connections	
Fibre optics cable	single mode Multi mode	100 Gbps	Advantages: • High speed • High band, width • High security • Long distance. Disadvantages: • Expensive • Require skilled installers.	Maximum distance of fibre optics cable is around 100 metres	

EXP. NO: 03 EXPERIMENTS ON CISCO PACKET
30/07/2026 TRACER (simulation Tool)

AIM:

To study the packet tracer tool installation and User interface overview.

c) To understand environment of CISCO PACKET TRACER to design simple network.

INTRODUCTION:

A simulator, as the name suggests, simulates network devices and its environments. Packet Tracer is an exciting network design, simulation and modelling tool.

1. It allows you to model complex systems, without the need for dedicated equipment.

2. It helps you to practice your network configuration and troubleshooting skills via compu-ter or an Android or iOS based mobile device.

3. It is available for both the Linux and Windows desktop environments.

4. Protocols in ~~panis~~ Packet Tracer are coded to work and behave in the same way as they would on real hardware.

INSTALLING PACKET TRACER:

To download Packet Tracer, go to https://www.netacad.com and login with your Cisco Networking Academy credentials; then, click on the Packet Tracer graphic and download the package appropriate for your operating system.

WINDOWS

Installation in windows is pretty simple and straightforward; the setup comes in a single

file named PacketTracer_setup6.0.1.exe. Open the file to begin the setup wizard, accept the license agreement, choose a location, and start the installa-tion.

LINUX:

Linux users with an Ubuntu/Debian distribution should download the file for Ubuntu, and those using Fedora/Redhat/Centos must download the file for Fedora. Grant executable permission to this file by using chmod, and execute it to begin the installation. chmod +x, PacketTracer601_i386-installer-rpm.bin.

./PacketTracer601_i386-installer-rpm.bin

USER INTERFACE OVERVIEW:

The Layout of the Packet Tracer is divided into several components. The components of the Packet Tracer interface are as follows: match the numbering with explanations.

1. Menu bar - This is common menu found in all software applications; it is used ~~as~~ to open, save, print, change preferences, and so on.

2. Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed such as open, save, zoom, undo, and redo, and on the right-hand side is an icon for entering network information for the current network.

3. Logical/physical workspace tabs - These tab allow you to toggle between the Logical and physi-cal work areas.

4. Workspace - This is the area where the topologies are created and simulations are displayed

5. Common tools bar - This toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete, inspect, resize shape, and add simple/complex PDU.

6. Real-time / simulation tabs - These tabs are used to toggle between the real and simulation modes. Buttons are also provided to control the time, and to capture the packets.

7. Network component box - This component contains all of the network and end devices available with Packet Tracer, and is further divided into two areas: Area 7a: Device-type selection box - This area contains device categories Area 7b: Device-specific selection box - when a device category is selected, this selection box displays the different device models within that category.

8. User-created packet box - Users can create highly-customized packets to test their topology from this area, and the results are displayed as a list.

d) Analyse the behaviour of network devices using CISCO PACKET TRACER simulator.

1. From the network, component box, click and drag and drop the below components:
   a) 4 Generic PCs and one HUB.
   b) 4 Generic PCs and one switch.
2. Click on connections:
   a) Click on copper-straight through cable.
   b) Select one of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly, connect the remaining 3 PCs to the HUB
   c) Similarly, connect 4 PCs to the switch using copper straight-through cable.
3. Click on the PCs connected to hub, go to the Desktop tab, click on the IP configuration, and

enter an IP address and subnet mask. Here, the default gateway and DNS server information is not needed as these are only two end devices in the network.

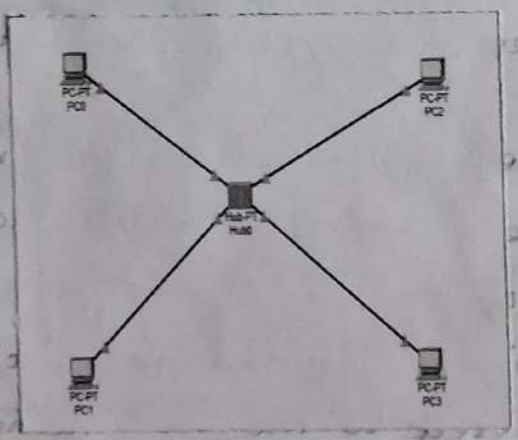
Click on the PDU (message icon) from the common tool bar,

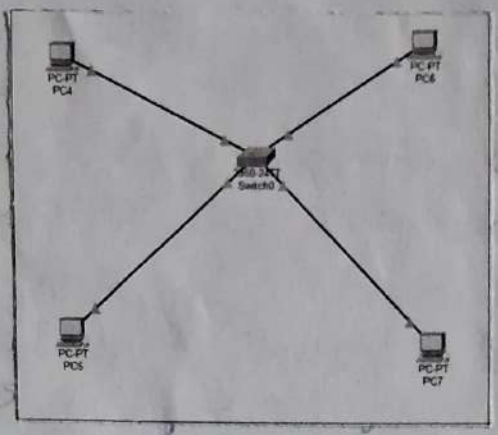
a) Drag and drop it on one of PC (source machine) and then drop it on another PC connected to the HUB.

4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

5. Repeat step #3 to step #5 for the PCs connected to the switch.

6. Observe how HUB and switch are forwarding the PDU and write your observation & conclusion about the behaviours of switch and HUB.





Student observation:

a) From your observation write down the behaviour of Switch and HUB in terms of forwarding the packets received by them.

Behaviour of HUB:

→ The HUB forwards every packet it receives to all connected ports, except the port on which the packet arrived.

→ All PCs connected to the HUB, receive the Packet, even though only the destination PC accepts it.

→ This results in unnecessary network traffic because every device sees the transmitted data.

Behaviour of Switch:

→ The Switch forwards the packet only to the port connected to the destination device after learning the MAC addresses of connected devices.

→ Other PCs connected to the switch do not receive the packet.

→ This reduces unnecessary traffic and makes communication more efficient and secure than a HUB.

b) Find out the network topology implemented in your college and draw and label that topology in your observation book.

→ Star topology is the network topology that is implemented in my college.

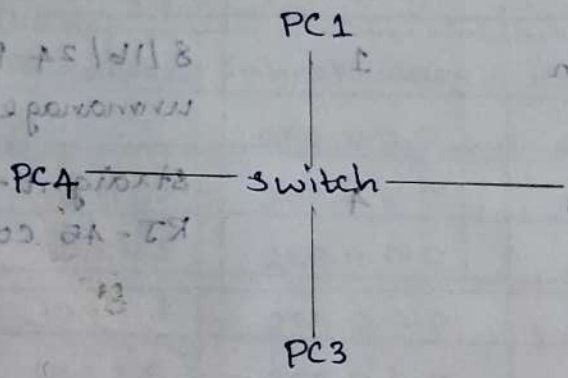
→ All computers and network devices are connected to a switch.

→ Communication between devices takes place through the switch.

→ If one cable or computer fails, the rest of the network continues to function normally.

→ Star topology is easy to manage, troubleshoot, and expand.

Diagram:



Result:

Thus, the packet tracer tool installation and user interface overview has been studied.

03/08/2026 SETTING UP A LOCAL

EXP.NO:04 AREA NETWORK (LAN)

AIM:

To design, set up, and configure a local area Network (LAN) connecting four computers through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command, and demonstrate resource/data sharing across the network.

EQUIPMENT REQUIRED:

Equipment	Quantity	specification
Personal computers (PC)	4	Ethernet (NIC) enabled.
Network switch	1	8/16/24 port, unmanaged / managed.
Ethernet cables (Cat5e/Cat6)	4	straight-through, RJ-45 connectors.

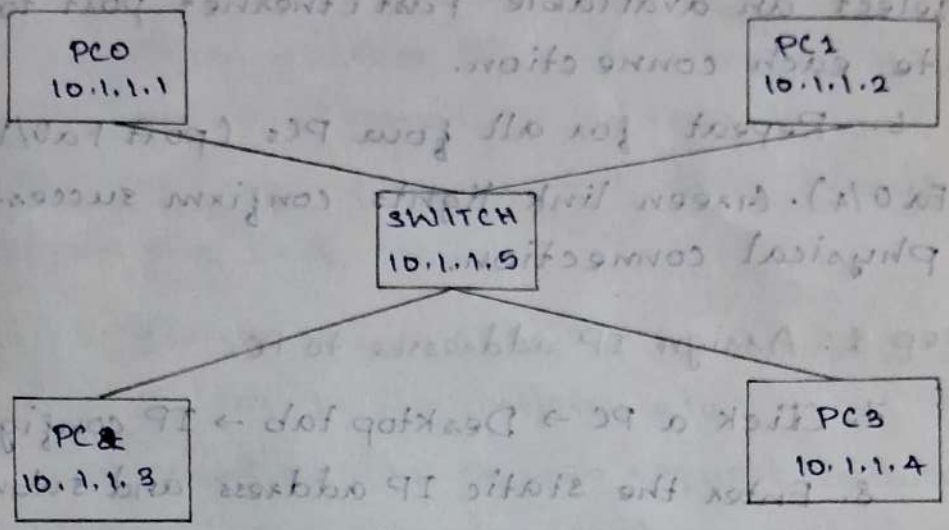
THEORY:

A Local Area Network (LAN) is a network that connects computers and devices within a limited geographical area such as a home, office or lab, allowing them to share resources such as files, Printers, and internet connections.

A star topology is used in this experiment, where all end devices (PCs) are individually connected to a central network switch. The switch operates at layer 2 of the OSI model and forwards data frames between connected devices based on their MAC addresses.

Network diagram:

The LAN was implemented using a star topology as shown below, with 4 PCs connected individually to a central switch.



IP Address configuration table:

Device	IP Address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch(Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface.

Implementation in Cisco Packet Tracer:

Since physical hardware was limited, the LAN was also implemented and verified using Cisco Packet Tracer simulation software, following the steps below.

Step 1: Place the devices

1. Open Cisco Packet Tracer
2. From the device category, panel, select end devices and drag 4 PCs on to the workspace.
3. Select switches and drag 1 switch onto the workspace.

Step 2: Cable the devices.

4. Click connections → select copper straight-Through cable.

5. Click each PC, then click the switch, and select an available Fast Ethernet port to comple-te each connection.

6. Repeat for all four PCs (port Fa0/1 to Fa0/4). Green link lights confirm successful Physical connection.

Step 3: Assign IP addresses to PCs

7. Click a PC → Desktop tab → IP configuration.

8. Enter the static IP address and subnet mask as per the IP table for each PC.

Step 4: Configure the Switch Management IP

9. Click the switch → CLI tab

10: Enter the following commands to assign a management IP:

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5.255.0.0.0
no shutdown
exit.
```

Step 5: Verify connectivity using ping.

11. Click a PC → Desktop tab → Command prompt

12. Use the ping command to test connectivity to the other PCs, e.g., ping 10.1.1.2.

13, successful "Reply from..." messages confirm connectivity.

Step 6: Demonstrate Data sharing.

14. Add a server device (End devices → server

PT) and connect it to the switch.

15. Assign it an IP address via its Desktop → IP configuration tab.

16. Open the server's services (config) tab and enable the HTTP service.

17. From another PC, open the Web Browser and enter the server's IP address to load the page, confirming successful data transfer across the LAN.

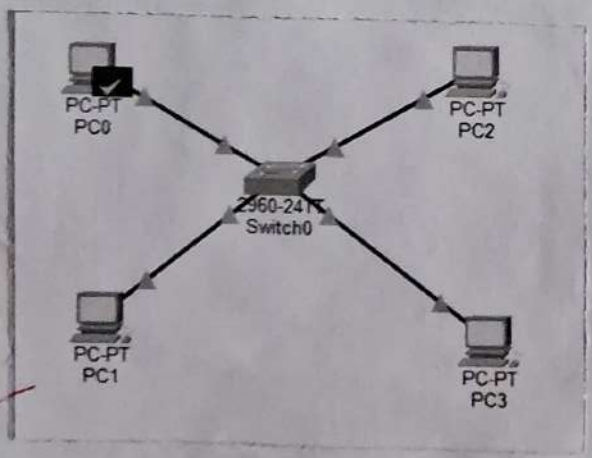
Step 7: Observe Packet flow in simulation mode.

18. Switch to Simulation mode.

19. Send a ping or web request between two devices and Use the play/capture controls to step through the packets.

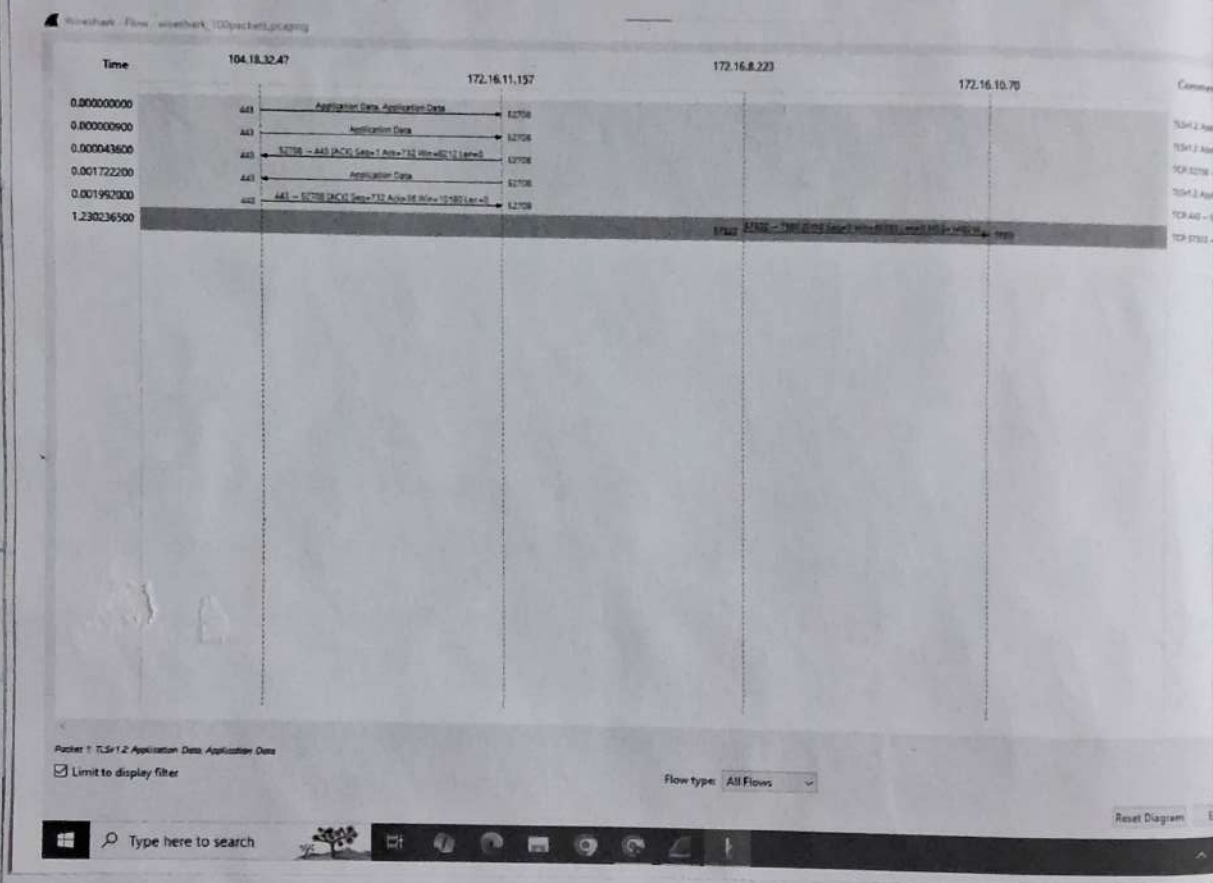
OBSERVATION:

All four PCs were successfully connected to the switch in a star topology and assigned unique static IP addresses within the same subnet (10.0.0.0/255.0.0.0). Ping tests between all devices returned successfully.



RESULT:

Thus, the LAN is configured using a switch in the network.


Time
104.18.32.47
172.16.11.157
172.16.8.223
172.16.10.70
0.000000000
0.000000900
0.000043600
0.001722200
0.001952000
1.230236500
Application Data, Application Data
Application Data
Application Data
Packet 1: TLSv1.2 Application Data, Application Data
Limit to display filter
Flow type: All Flows
Reset Diagram
Type here to search

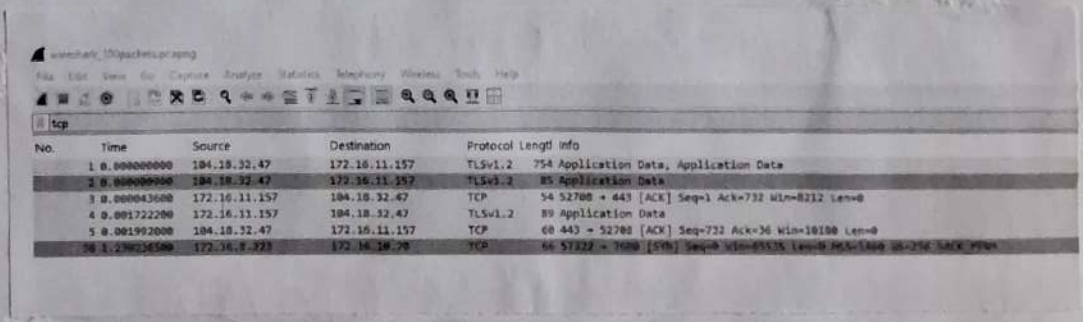
EXP. NO: 05 EXPERIMENTS ON PACKET
10/08/2026 CAPTURE TOOL : WIRESHARK

AIM :

To capture, filter, and always analyze the network packets using the wireshark packet capture tool, and to study different network protocols such as TCP, UDP, ARP, DNS, HTTP, IP, ICMP, and DHCP by inspecting their packet details and flow graphs.

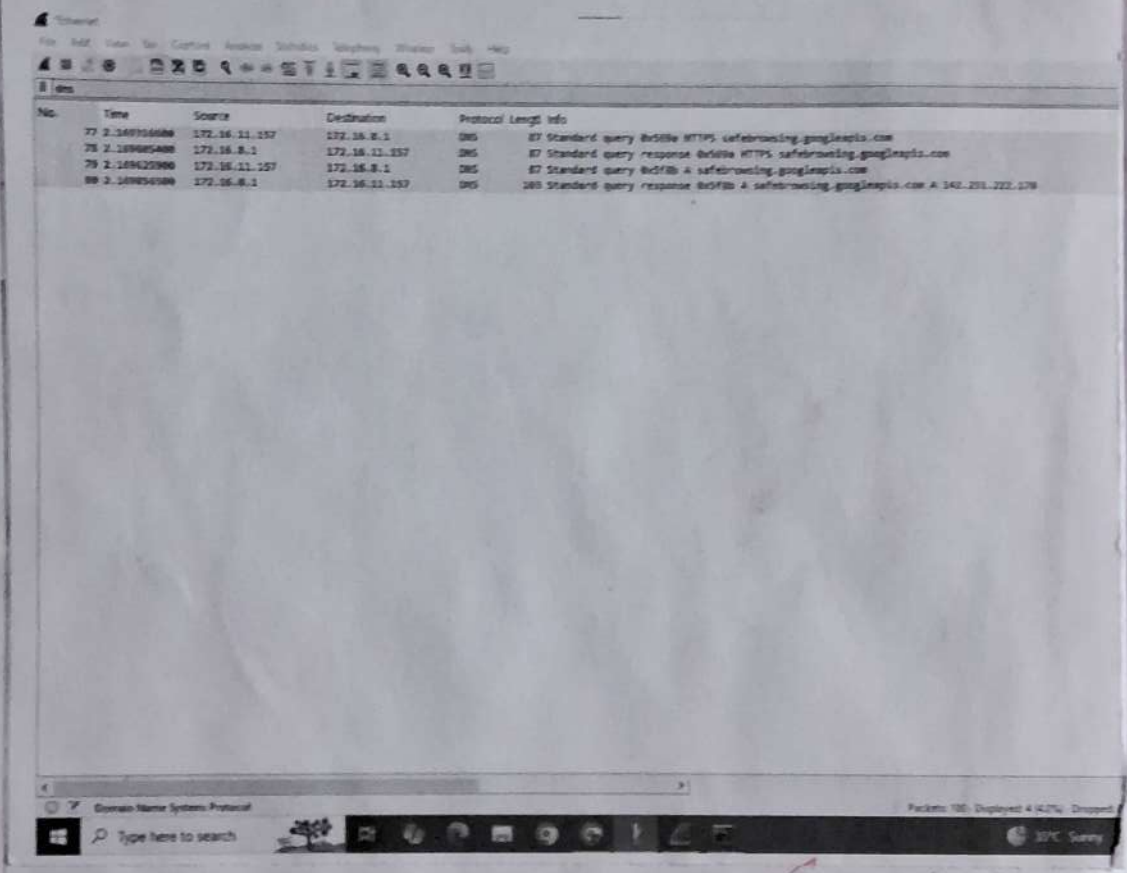
WIRESHARK :

Wireshark, a network analysis tool formerly known as ethereal, captures packets in real time and display them in human-readable format. It includes filters, colors, coding, and other features that let you dig deep into network traffic and inspect individual packets.



RESULT:

Thus, TCP, UDP, ARP, DNS, HTTP, IP, ICMP and DHCP are studied by inspecting their Packet details and flow graphs.


dns
No. Time Source Destination Protocol Length Info
77 2.169716600 172.16.11.157 172.16.8.1 DNS 87 Standard query 0x58be HTTPS safebrowsing.googleapis.com
78 2.169805400 172.16.8.1 172.16.11.157 DNS 87 Standard query response 0x58be HTTPS safebrowsing.googleapis.com
79 2.169823900 172.16.11.157 172.16.8.1 DNS 87 Standard query 0x5f8b A safebrowsing.googleapis.com
80 2.169850500 172.16.8.1 172.16.11.157 DNS 103 Standard query response 0x5f8b A safebrowsing.googleapis.com A 142.251.222.170
Domain Name System: Protocol
Packets: 100 · Displayed: 4 (4.0%) · Dropped:
Type here to search
31°C Sunny

EXP. NO: 05 EXPERIMENTS ON PACKET

internal

arp

No.	Time	Source	Destination	Protocol	Length	Info
8	0.082057300	Dell_37:fa:af	Broadcast	ARP	60	Who has 172.16.9.180? Tell 172.16.9.184
14	0.204218100	MicroStarINT_c5:cd:...	Broadcast	ARP	60	Who has 172.16.11.150? Tell 172.16.10.13
15	0.328338600	HP_5c:06:39	Broadcast	ARP	60	Who has 172.16.10.164? Tell 172.16.10.163
19	0.373334800	HP_5b:db:ac	Broadcast	ARP	60	Who has 172.16.10.174? Tell 172.16.10.144
22	0.449429500	HP_5b:dd:f0	Broadcast	ARP	60	Who has 172.16.10.74? Tell 172.16.10.165
24	0.539647300	MicroStarINT_c5:cB:...	Broadcast	ARP	60	Who has 172.16.10.245? Tell 172.16.8.240
25	0.671855700	Sophos_cf:be:45	Broadcast	ARP	60	Who has 172.16.11.212? Tell 172.16.8.1
28	0.745608700	MicroStarINT_c5:cb:...	Broadcast	ARP	60	Who has 172.16.9.133? Tell 172.16.10.9
29	0.784633900	EliteGroupCo_14:8a:...	Broadcast	ARP	60	Who has 172.16.9.34? Tell 172.16.10.199
30	0.843063400	MicroStarINT_cc:27:...	Broadcast	ARP	60	Who has 172.16.9.133? Tell 172.16.9.66
32	0.944818900	HP_5b:dd:f0	Broadcast	ARP	60	Who has 172.16.11.5? Tell 172.16.10.165
33	0.966432100	ASUSTekCOMPU_99:99:...	Broadcast	ARP	60	Who has 172.16.8.28? Tell 172.16.8.171
34	1.003725900	Dell_37:fa:af	Broadcast	ARP	60	Who has 172.16.9.180? Tell 172.16.9.184
35	1.043919400	HP_5b:db:ac	Broadcast	ARP	60	Who has 172.16.10.174? Tell 172.16.10.144
36	1.188887700	EliteGroupCo_15:eb:...	Broadcast	ARP	60	Who has 172.16.10.164? Tell 172.16.10.191
37	1.203872600	MicroStarINT_c5:cd:...	Broadcast	ARP	60	Who has 172.16.11.150? Tell 172.16.10.13
39	1.272970700	MicroStarINT_c5:ce:...	Broadcast	ARP	60	Who has 172.16.10.60? Tell 172.16.10.53
40	1.317212200	HP_5c:06:39	Broadcast	ARP	60	Who has 172.16.10.164? Tell 172.16.10.163
41	1.368523500	Dell_35:11:a6	Broadcast	ARP	60	Who has 172.16.10.67? Tell 172.16.9.194
60	1.436934000	HP_5b:ce:b4	Broadcast	ARP	60	Who has 172.16.10.245? Tell 172.16.10.137
71	1.442230200	HP_5b:dd:f0	Broadcast	ARP	60	Who has 172.16.10.74? Tell 172.16.10.165
74	1.605838900	Sophos_cf:be:45	Broadcast	ARP	60	Who has 172.16.11.212? Tell 172.16.8.1
75	1.646576900	MicroStarINT_c5:ce:...	Broadcast	ARP	60	Who has 172.16.9.180? Tell 172.16.10.27
80	1.723210200	HP_5c:0b:2f	Broadcast	ARP	60	Who has 172.16.10.153? Tell 172.16.10.154
81	1.729071300	EliteGroupCo_14:8a:...	Broadcast	ARP	60	Who has 172.16.9.34? Tell 172.16.10.199
90	1.851992300	EliteGroupCo_15:ec:...	Broadcast	ARP	60	Who has 172.16.9.168? Tell 172.16.10.205
92	1.927709100	EliteGroupCo_14:8a:...	Broadcast	ARP	60	Who has 172.16.9.78? Tell 172.16.10.179
93	1.984866700	HP_5b:dd:f0	Broadcast	ARP	60	Who has 172.16.11.5? Tell 172.16.10.165
94	2.002119500	Dell_37:fa:af	Broadcast	ARP	60	Who has 172.16.9.180? Tell 172.16.9.184

Address Resolution Protocol: Protocol

Type here to search

```
.1
.1
t DESKTOP-T0S13VS, Workstation, Server, NT Workstation
.1
.1
t DESKTOP-8RADVTB, Workstation, Server, NT Workstation
x683a ANY MDC0422280
x683a ANY MDC0422280
1
1
1.1
1
1
t DESKTOP-ELQ2B13, Workstation, Server, NT Workstation
x0000 ANY DESKTOP-8C1RML7.local, "QM" question
x0000 ANY DESKTOP-8C1RML7.local, "QM" question
xc747 ANY DESKTOP-8C1RML7
xc747 ANY DESKTOP-8C1RML7
x0000 ANY DESKTOP-8C1RML7.local, "QM" question
x0000 ANY DESKTOP-8C1RML7.local, "QM" question
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
0000 ANY DESKTOP-8C1RML7.local, "QM" question
0000 ANY DESKTOP-8C1RML7.local, "QM" question
15ed ANY DESKTOP-8C1RML7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
15ed ANY DESKTOP-8C1RML7
0000 ANY DESKTOP-8C1RML7.local, "QM" question
0000 ANY DESKTOP-8C1RML7.local, "QM" question
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
sponse 0x0000 AAAA fe80::b7b5:76e5:cd34:ed0d A 172.16.9.7
.1
DESKTOP-AI7U1DU, Workstation, Server, NT Workstation
0000 PTR _oculusal_sp_v2._tcp.local, "QM" question
sponse 0x0000 PTR Administrator::DESKTOP-5F4MIEQ._oculusal_sp_v2._tcp.local SRV 0 0 56044 DESKTOP-5...
0000 PTR _oculusal_sp_v2._tcp.local, "QM" question
sponse 0x0000 PTR Administrator::DESKTOP-5F4MIEQ._oculusal_sp_v2._tcp.local SRV 0 0 56044 DESKTOP-5...
0000 PTR _oculusal_sp._tcp.local, "QM" question
```

01 00 5e 7f ff fa 00 27 0e 13 f0 9e 08 00 45 00

Packets: 100 · Displayed: 48 (48.0%) · Dropped: 0 (0.0%)

Profile: Default

36°C Mostly sunny

ENG IN 3:58 PM 8/10/2026

EXP.NO:06

10/09/2026

HAMMING CODE

AIM:

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

CODE:

Sender:

```
text = input("Enter the text:")
binary = " "
for ch in text:
    binary += format(ord(ch),'08b)
print("Binary data:", binary)
channel = " "
for x in range(0,len(binary),8):
    d = binary[x:x+8]
    code = ['0']*13
    code[3] = d[0]
    code[5] = d[1]
    code[6] = d[2]
    code[7] = d[3]
    code[9] = d[4]
    code[10] = d[5]
    code[11] = d[6]
    code[12] = d[7]
    code[1] = str(int(code[3])^int(code[5])
        ^int(code[7])^int(code[9])^
            int(code[11]))
```

```
code[2] = str(int(code[3])^int(code[6])^
          int(code[7])^int(code[10])^
          int(code[11]))
code[4] = str(int(code[5])^int(code[6])^
          int(code[7])^int(code[12]))
code[8] = str(int(code[9])^int(code[10])^
          int(code[11])^int(code[12]))

for i in range(1,13):
    channel += code[i]
print("Hamming code:", channel)
with open("channel.txt","w") as f:
    f.write(channel)
```

Receiver:

```
with open("channel.txt","r") as f:
    received = f.read()
print("Received data:", received)
corrected = ""
for x in range(0, len(received), 12):
    c = ['0'] + list(received[x:x+12])
    r1 = (int(c[1])^int(c[3])^int(c[5])^int(c[7])^
          int(c[9])^int(c[11]))
    r2 = (int(c[2])^int(c[3])^int(c[6])^
          int(c[7])^int(c[10])^int(c[11]))
    r4 = (int(c[4])^int(c[5])^int(c[6])^
          int(c[7])^int(c[12]))
    r8 = (int(c[8])^int(c[9])^int(c[10])^
          int(c[11])^int(c[12]))
    error = r1*1 + r2*2 + r4*4 + r8*8
```

```
if error == 0:
    print("No error detected")

else:
    print("Error detected at position:", error)
    if c[error] == '0':
        c[error] = '1'
    else:
        c[error] = '0'
    print("Error corrected.")

corrected += (c[3] + c[5] + c[6] + c[7] + c[9]
              + c[10] + c[11] + c[12])

output = ""
for x in range(0, len(corrected), 8):
    byte = corrected[x : x+8]
    if len(byte) == 8:
        output += chr(int(byte, 2))
print("Original text:", output)
```

Output:

```
Enter the text: hello
Binary data: 01101000011001010110110001101111
             000110111
Hamming code: 010011011000100111000101
              000011001100000011001100
              110111001111
Hamming code saved in channel.txt.
Received data: 010001011000100111000101000
               001100110000001100110011011
               001111
Error detected at position: 5
Error corrected.
```

EXP.NO:07

17/09/2026

SLIDING WINDOW PROTOCOL

AIM:

Write a program to implement flow chart at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another.

CODE:

Sender:

```
import time
import os
window_size = int(input("Enter window size:"))
message = input("Enter the message:")
frames = []
for i, ch in enumerate(message, start=1):
    frames.append((i, ch))
sender_buffer = "sender_buffer.txt"
receiver_buffer = "Receiver_buffer.txt"
start = 0
while start < len(frames):
    window = frames[start : start + window_size]
    print("\nSender")
    with open(sender_buffer, "w") as file:
        for frame_no, data in window:
            file.write(f"{frame_no}, {data}\n")
            print("Frame", frame_no, ":", data)
    print("Frames written to sender_buffer.txt")
    input("\nPress enter to continue.")
    if not os.path.exists(receiver_buffer):
        print("Receiver_buffer.txt no found")
        continue
    with open(receiver_buffer, "r") as file:
        response = file.read().strip()
    print("Received:", response)
    if response.startswith("ACK:"):
        ack_no = int(response.split(":")[1])
        expected_ack = window[-1][0]
```

```
if ack_no == expected_ack:
        print("ACK", ack_no, "received")
        print("Window moved")
        start = start + window_size

    else:
        print("Wrong ACK")
        print("Expected:", expected_ack)
        print("Received", ack_no)
        print("Resending frames")

elif response.startswith("NACK:"):
    nack_no = int(response.split(":")[1])
    print("NACK", nack_no, "received")
    print("Resending from frames", nack_no)
    start = nack_no - 1

else:
    print("Invalid response")

time.sleep(1)
print("\nAll frames are sent")
```

Receiver:

```
import os
sender_buffer = "sender_Buffer.txt"
received_buffer = "Receiver_Buffer.txt"
expected_frame = int(input("Enter expected frame no:"))
print("\nReceiver")
if not os.path.exists(sender_buffer):
    print("sender_Buffer.txt not found")

else:
    with open(sender_buffer, "r") as file:
        lines = file.readlines()
    error = False
    last_frame = expected_frame - 1
    print("Frames received")
    for line in lines:
        line = line.strip()
        if line == "":
            continue
```

```
        parts = line.split(",")
        frame_no = int(parts[0])
        data = parts[1]
        print("Frame", frame_no, ":", data)
        if frame_no == expected_frame:
            print("Frame", frame_no, "received
                correctly")
            last_frame = frame_no
            expected_frame += 1

        else:
            print("Error in frames", frame_no)
            print("Expected frames:", expected_frame)
            error = True
            with open(receiver_buffer, "w") as file:
                file.write("NACK : {expected_frames}")
            print("NACK", expected_frame, "sent")
            break
    if not error:
        with open(receiver_buffer, "w") as file:
            file.write(f"ACK : {last_frame}")
        print("ACK", last_frame, "sent")
```

OUTPUT :

Sender (window 1):

```
Enter window size : 3
Enter the message : HELLO
 sender
Frame 1 : H
Frame 2 : E
Frame 3 : L
Frames written to sender_Buffer.txt
Received : ACK : 3
 Ack 3 received
 Window moved
```

Sender

Frame 4 : L

Frame 5 : O

Frames written to sender-Buffer.txt

Received : ACK : 5

ACK 5 received

Window moved.

All frames sent successfully.

Receiver (window 2) :

Expected frame no : 1

Receiver

Frame 1 : H

received correctly

Frame 2 : E

received correctly

Frame 3 : L

received correctly

ACK 3 sent

Expected frame no : 4

Receiver

Frame 4 :

received correctly

Frame 5 :

received correctly.

RESULT :

Thus, the frames are sent and received using the sliding window protocol.

EXP.NO:08 (a) NMAP TO DISCOVER LIVE HOSTS
28/09/2026 USING NMAP SCANS ON
THE TRYHACKME PLATFORM

AIM:

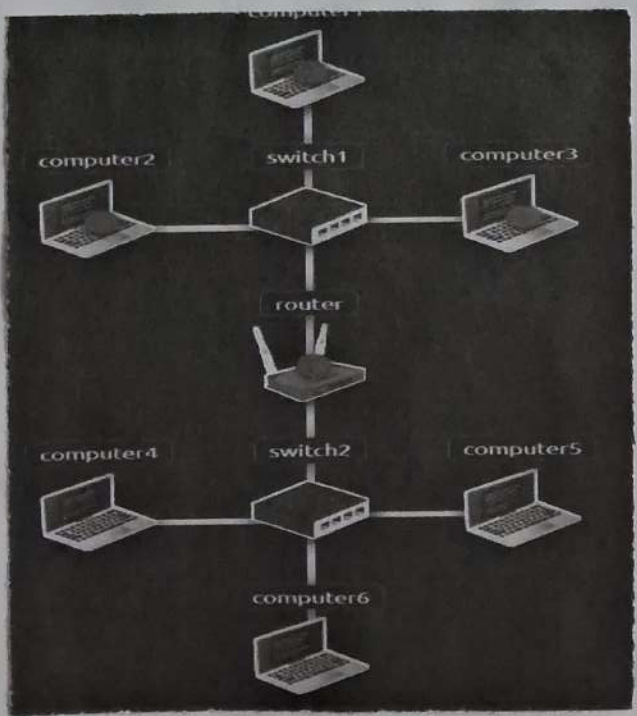
To discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform.

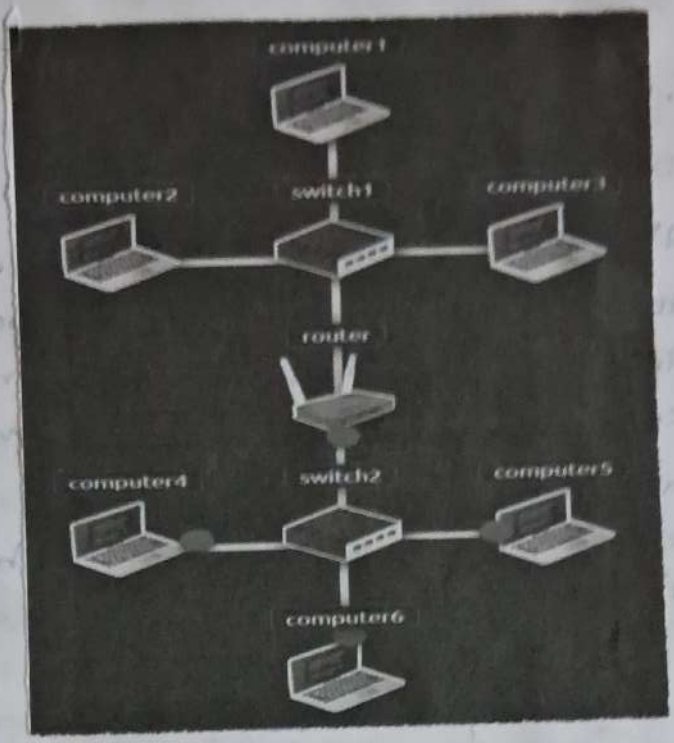
INTRODUCTION:

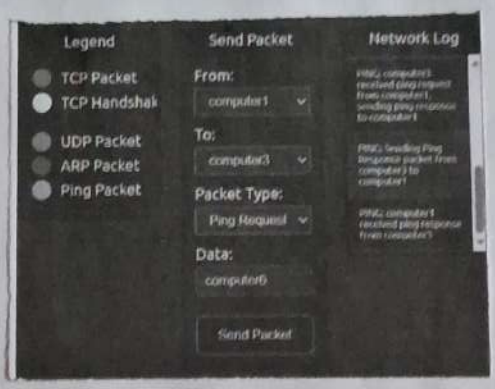
When targetting a network, we need an efficient tool to handle repetitive tasks. This tool should help us find out which systems are active and what services are running on those systems.

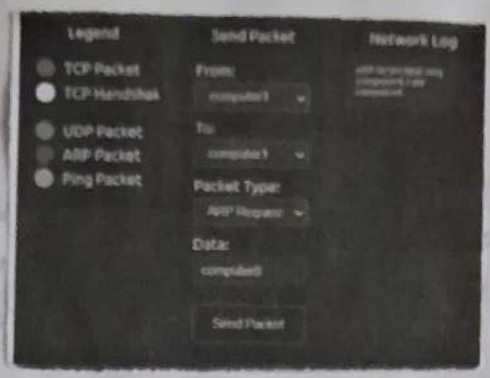
This room is the first of four in this Nmap series. These four rooms are also part of the Network security module.

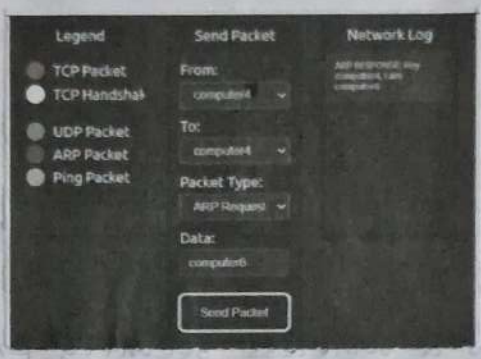
- Nmap Live Host Discovery
- Nmap Basic Port scans
- Nmap Advanced Port scans
- Nmap Post Port scans

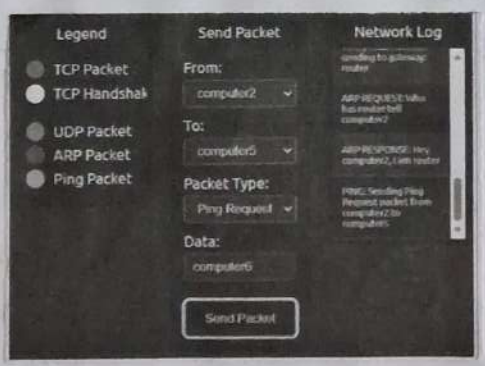



computer1
computer2
switch1
computer3
router
computer4
switch2
computer5
computer6


Legend
TCP Packet
TCP Handshak
UDP Packet
ARP Packet
Ping Packet
Send Packet
From:
computer1
To:
computer3
Packet Type:
Ping Request
Data:
computer6
Send Packet
Network Log







RESULT :

Thus, live hosts are successfully identified using Nmap ARP scanning.

EXP.NO:08 (b) CONFIGURATION OF WLAN
28/09/2026 USING CISCO PACKET TRACER

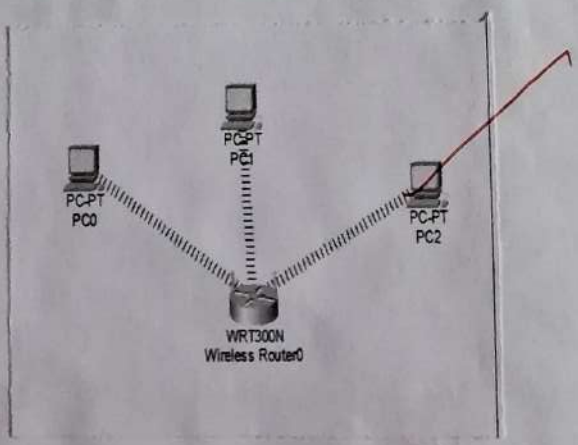
AIM:

To configure a Wireless LAN using cisco packet tracer and establish wireless communication between computers through a wireless router.

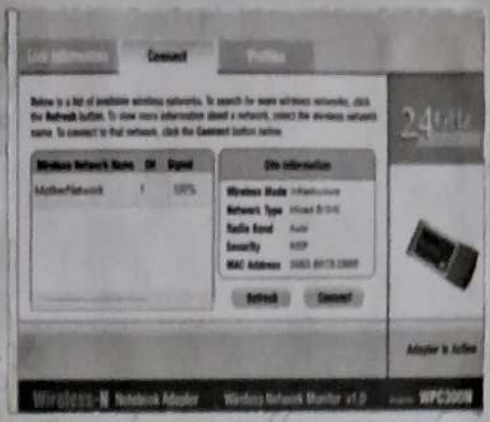
STEPS:

1. SSID is the name used to identify a wireless network.
2. Security key is a password used to protect the wireless network from unauthorized access.
3. Wireless LAN connects device wirelessly using a wireless router or access point.
4. Static IP address is manually assigned to each device for network communication.
5. WEP is a wireless security protocol used to protect the network.
6. The configured SSID for this experiment is MotherNetwork.
7. The WEP key used is 0123456789.

TOPOLOGY:



OUTPUT:



RESULT:

Thus, a secure Wireless LAN was successfully configured using Cisco Packet Tracer, and all three PCs were connected to the wireless router.

EXP.NO:09
28/09/2026

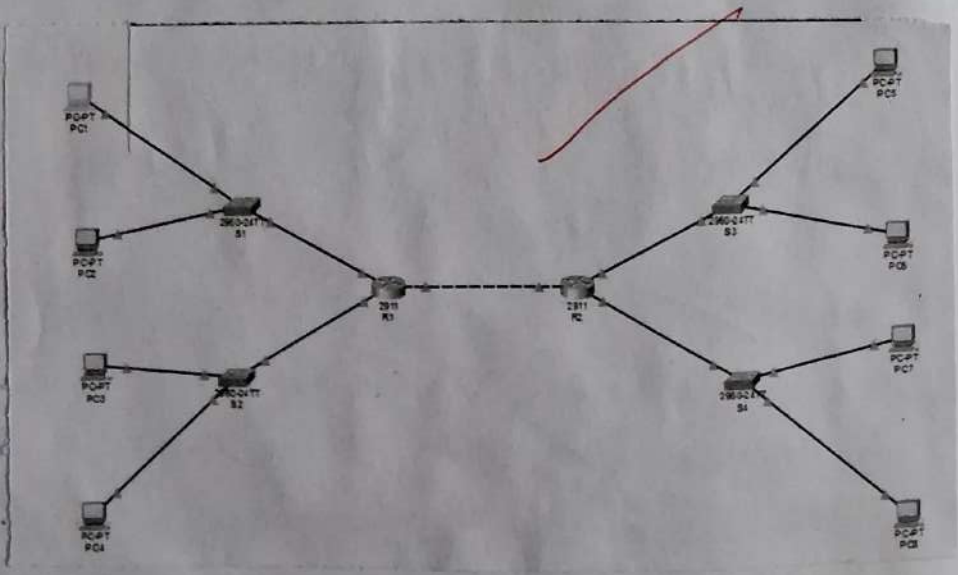
IMPLEMENTATION OF SUBNETTING USING THE CISCO PACKET TRACER

AIM:

To implement classless IP subnetting and configure communication between different subnets using cisco packet tracer.

STEPS:

1. subnetting is the process of dividing a large IP network into smaller networks called subnets.
2. Classless subnetting uses a variable-length subnet mask instead of relying only on the default class based-masks.
3. In this experiment, the network 192.168.1.0/24 is dividing using the /27 subnet mask.
4. The subnet mask for /27 is 255.255.255.224
5. A /27 subnet provides 32 total addresses and 30 usable host addresses.
6. Subnetting helps in efficient IP address utilization and reduces broadcast traffic.
7. Different subnets can communicate through a router.



RESULT:

Thus, a classless IP subnetting was successfully implemented in cisco packet tracer, and connectivity between the different subnets was verified using the ping command.

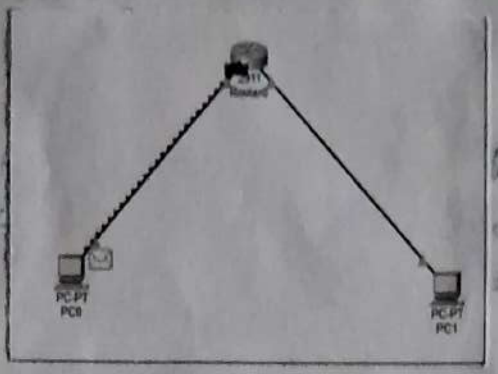
EXP.NO: 10 (a) Internetworking with routers
28/09/2026 in cisco packet tracer.

AIM:

To design and configure a simple networking using a router and establish communication between two different networks.

PROCEDURE:

1. Open cisco packet tracer and place one router and two PCs in the workspace.
2. Connect PC0 to R1 GigabitEthernet0/0 and PC1 to R1 GigabitEthernet0/1 using copper-straight-through cables.
3. Configure R1 G0/0 with:
   IP: 192.168.10.1
   subnet Mask: 255.255.255.0
4. Configure R1 G0/1 with:
   IP: 192.168.20.1
   subnet mask: 255.255.255.0
5. Enable both router interfaces using no shutdown command.
6. Configure PC0 with:
   IP: 192.168.10.2
   subnet mask: 255.255.255.0
   Gateway: 192.168.10.1
7. Configure PC1 with:
   IP: 192.168.20.2
   Subnet mask: 255.255.255.0
   Gateway: 192.168.20.1
8. Test connectivity by pinging from PC0 to PC1 using: ping 192.168.20.2
9. Switch to simulation mode & send a simple PDU from PC0 to PC1.




```
exit
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.10.1    YES manual up                    up
GigabitEthernet0/1     192.168.20.1    YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
Router#


Router con0 is now available
```

RESULT:

Thus, a simple internetwork was successfully configured using a router, and communication between PC0 and PC1 was successfully verified.

EXP.NO:10
28/09/2026

(b) DESIGN AND CONFIGURE AN INTERNETWORK USING WIRELESS ROUTER, DHCP SERVER AND INTERNET CLOUD

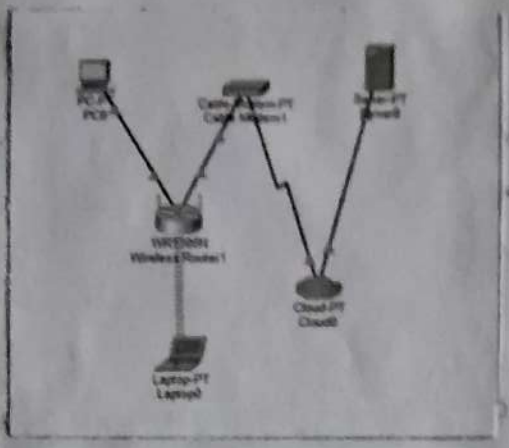
AIM:

To design and configure an internetwork using wireless router, DHCP server and internet cloud in cisco packet tracer.

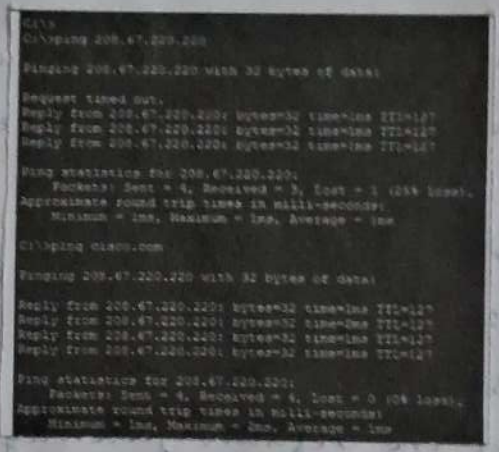
PROCEDURE:

1. Create the topology using a PC, laptop, wireless router, cable modem, internet cloud & cisco.com server.
2. Connect the devices using appropriate cable.
3. Configure the wireless router using the SSID HomeNetwork and enable DHCP
4. Configure the laptop to connect to the wireless network.
5. Configure PC0 to obtain its IP address automatically using DHCP.
6. Configure the internet cloud and cable modem connections.
7. Configure the cisco.com server as a DHCP and DNS server.
8. Assign the server IP address 208.67.220.220
9. Verify the IP configuration of PC0 using ipconfig /all.
10. Test connectivity using ping 208.67.220.220 and ping cisco.com.

TOPOLOGY :



OUTPUT :



RESULT : 8/11

Thus, an internetwork using a wireless router, DHCP server and internet cloud was successfully configured, and connectivity to the cisco.com server was verified.

EXP.NO:10 (c) DESIGNING AN INTER-NETWORK
28/09/2026 USING SWITCH, ROUTER & ETHERNET

AIM:

To design and configure an internetwork using a switch, router, and ethernet cables in cisco packet tracer.

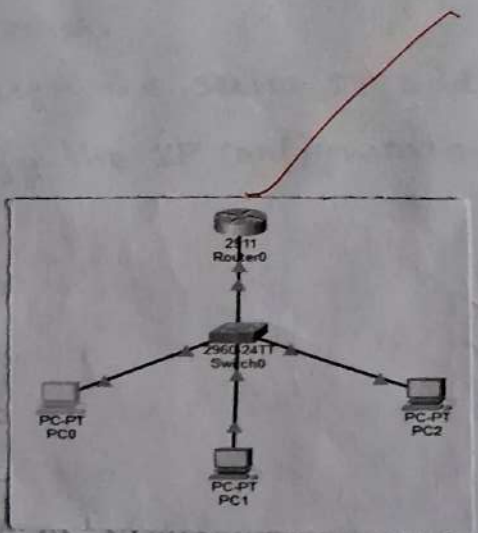
THEORY:

A switch connects multiple devices within a LAN, while a router connects and manages communication between networks. Ethernet cables are used to establish wired connections between the devices.

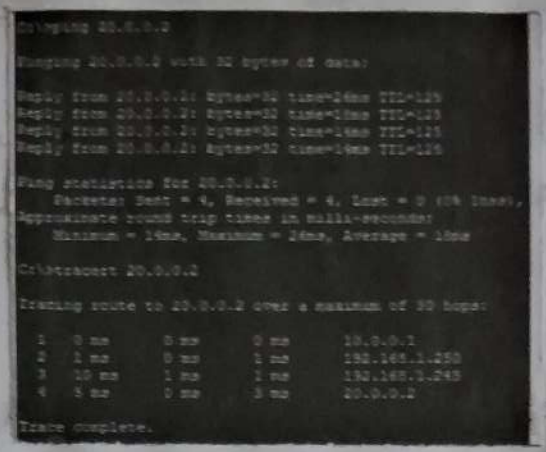
PROCEDURE:

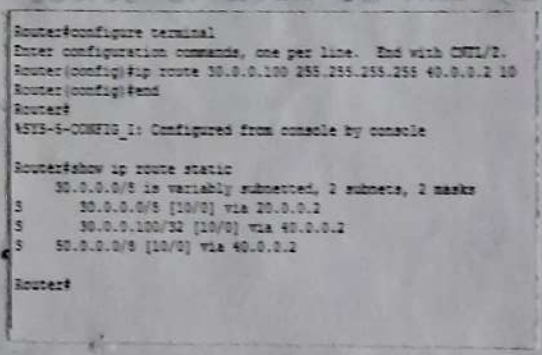
1. Place 1 router, 1 switch and 3 PCs.
2. Connect them using copper straight-through cables.
3. Configure router G0/0 as 192.168.30.1/24.
4. Assign PC0, PC1, and PC2 the IPs 192.168.30.2, .3 and .4.
5. Set the default gateway as 192.168.30.1.
6. Use ping to verify connectivity between the PCs.

TOPOLOGY:



OUTPUT :





RESULT:

Thus, the internetwork was successfully configured using a switch, router and ethernet cables, and connectivity was verified using the ping command.

EXP.NO: 11
29/09/2026

(a) STATIC ROUTING CONFIGURATION USING CISCO PACKET TRACER

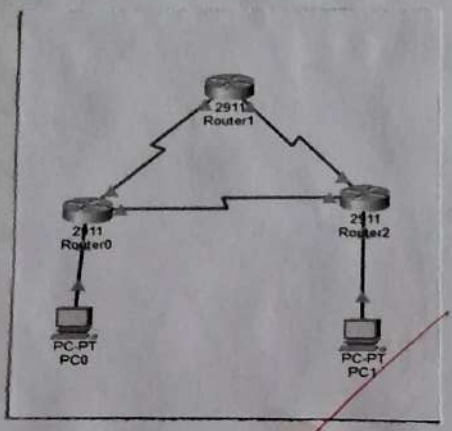
AIM:

To simulate static routing configuration using Cisco packet tracer and verify the main and backup routes.

THEORY:

Static routing is a routing method in which routes are manually configured in the router's routing table. A static route specifies the destination network, subnet mask, and next-hop address. Administrative distance (AD) can be used to select a preferred route and configure a backup route. A route with a lower AD is preferred over a route with a higher AD.

TOPOLOGY:



IP ADDRESSING TABLE:

Device	Interface	IP Address
PC0	Fast Ethernet0	10.0.0.2
Router0	G0/0	10.0.0.1
Router0	S0/3/0	20.0.0.1
Router1	S0/3/0	20.0.0.2
Router1	S0/3/1	50.0.0.1
Router2	S0/3/0	50.0.0.2
Router0	S0/3/1	40.0.0.1
Router2	S0/3/1	40.0.0.2
Router2	G0/0	30.0.0.1
PC1	FastEthernet0	30.0.0.100

subnet masks:

/8 - 255.0.0.0

/32 - 255.255.255.255

PROCEDURE:

1. Create the network topology using three routers and two PCs in cisco packet tracer.
2. Configure the IP addresses of the PCs and router interfaces
3. Enable the router interfaces using the no shut down command.
4. Configure static routes on Router0, Router1, and Router2.
5. Configure preferred and backup routes using different administrative distance values.
6. Verify the routes using show ip route static.
7. Test connectivity using ping.
8. Use tracert to verify the path followed by packets.

9. Disconnect the main route between Router0 and Router1.

10. Verify that the backup route is automatically selected.

ROUTING:

```
FastEthernet0 Connection:(default port)

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.....: FE80::290:2BFF:FE7E:8E4C
   IPv6 Address.....: ::
   IPv4 Address.....: 192.168.1.11
   Subnet Mask.....: 255.255.255.224
   Default Gateway.....: ::
                         192.168.1.1

Bluetooth Connection:

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.....: ::
   IPv6 Address.....: ::
   IPv4 Address.....: 0.0.0.0
   Subnet Mask.....: 0.0.0.0
   Default Gateway.....: ::
                         0.0.0.0

C:\>ping 192.168.2.11

Pinging 192.168.2.11 with 32 bytes of data:

Reply from 192.168.2.11: bytes=32 time<1ms TTL=127
Reply from 192.168.2.11: bytes=32 time<1ms TTL=127
Reply from 192.168.2.11: bytes=32 time<1ms TTL=127
Reply from 192.168.2.11: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.2.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 192.168.3.11

Pinging 192.168.3.11 with 32 bytes of data:

Reply from 192.168.3.11: bytes=32 time<1ms TTL=126
Reply from 192.168.3.11: bytes=32 time=14ms TTL=126
Reply from 192.168.3.11: bytes=32 time<1ms TTL=126
Reply from 192.168.3.11: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.3.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 14ms, Average = 3ms

C:\>ping 192.168.4.11

Pinging 192.168.4.11 with 32 bytes of data:

Reply from 192.168.4.11: bytes=32 time<1ms TTL=126
Reply from 192.168.4.11: bytes=32 time=1ms TTL=126
Reply from 192.168.4.11: bytes=32 time=1ms TTL=126
Reply from 192.168.4.11: bytes=32 time=1ms TTL=126

Ping statistics for 192.168.4.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

RESULT:

Thus, static routing was successfully simulated using cisco packet tracer. The main and backup static routes were configured and verified. The backup route was automatically selected when the ~~mai~~ primary route failed.

EXP.NO:11 (b) RIP USING CISCO PACKET TRACER

29/09/2026

AIM:

To implement dynamic routing using RIP in cisco packet tracer and verify automatic route selection and rerouting.

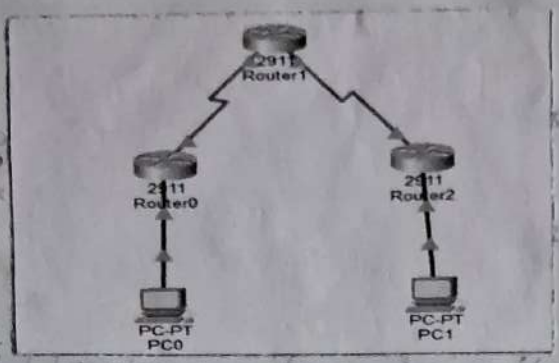
THEORY:

RIP (Routing information protocol) is a distance-vector protocol that uses hop count to select routes. It automatically updates routing information when the network topology changes.

PROCEDURE:

1. Create a network with 3 routers and 2 PCs.
2. Configured IP addresses on PCs and router interfaces.
3. Enabled all required route interfaces using no shutdown.
4. Configured RIP using the router rip command and added the connected networks.
5. Tested connectivity between PC0 and PC1 using ping.
6. Used Tracert to verify the route selected by RIP.
7. Disconnected the direct Router0-router2 link.
8. Verified that RIP automatically selected the alternate path through Router1.

TOPOLOGY :



OUTPUT :

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.30.1

Pinging 192.168.30.1 with 32 bytes of data:

Reply from 192.168.30.1: bytes=32 time=1ms TTL=255
Reply from 192.168.30.1: bytes=32 time<1ms TTL=255
Reply from 192.168.30.1: bytes=32 time<1ms TTL=255
Reply from 192.168.30.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.30.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>ping 192.168.30.3

Pinging 192.168.30.3 with 32 bytes of data:

Reply from 192.168.30.3: bytes=32 time<1ms TTL=128
Reply from 192.168.30.3: bytes=32 time<1ms TTL=128
Reply from 192.168.30.3: bytes=32 time<1ms TTL=128
Reply from 192.168.30.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.30.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 192.168.30.4

Pinging 192.168.30.4 with 32 bytes of data:

Reply from 192.168.30.4: bytes=32 time<1ms TTL=128
Reply from 192.168.30.4: bytes=32 time<1ms TTL=128
Reply from 192.168.30.4: bytes=32 time<1ms TTL=128
Reply from 192.168.30.4: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.30.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

RESULT :

Thus, RIP dynamic routing was successfully implemented in cisco packet tracer, and automatic route selection and rerouting were verified.

08/10/2026
EXP. NO:12

ECHO CLIENT - SERVER USING TCP/UDP SOCKETS.

AIM:

To implement an echo client-server using TCP/UDP sockets.

CODE :(TCP)

```
import socket
import threading
def start_server():
    server = socket.socket(socket.AF_INET,
             socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    Print("[SERVER] Listening on 127.0.0.1:5000")
    conn,addr = server.accept()
    Print(f"[SERVER] connected by {addr}")
    While True:
        data = conn.recv(1024).decode()
        if not data or data.lower()=='quit':
            print("[SERVER] connection closed
                  by client.")
            break
        print(f"[SERVER] Received: {data}")
        pconn.send(f"server received: {data}".
            encode()
    conn.close()
    server.close()
def start_client():
        client = socket.socket(socket.AF_INET,
                 socket.SOCK_STREAM)
        client.connect(('127.0.0.1',5000))
        Print("[CLIENT] connected to server")
        While True:
            msg = input("Enter message or('quit'
                  to exit):")
```

```
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
    client.close()
if __name__ == "__main__":
    server_thread = threading.Thread(target=start_
                    server, daemon=True)
    server_thread.start()
    start_client()
```

OUTPUT:

```
Enter message(or 'quit' to exit): hello
Received: hello
Server received: hello
Enter message (or 'quit' to exit: quit
connection closed by client.
```

CODE: (UDP)

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_
                      DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    While True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] Client {addr} ended the
                  connection.")
            break
        print(f"[SERVER] Received from {addr}: {msg}
        s.sendto(data, addr)
    s.close()
    Print("[SERVER] stopped.")
```

```
def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGR
        -AM)
    server = ('127.0.0.1', 12000)
    While True:
        msg = input("Enter message (or 'quit' to
            stop):")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] closing connection')
            break
        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.decode())
    c.close()
    print("[CLIENT] stopped.")
if __name__ == "__main__":
    threading.Thread(target = udp_server,
        daemon = True).start().udp_client()
```

OUTPUT:

```
Enter message (or 'quit' to stop): [SERVER] Running on
127.0.0.1:12000 hi
Received from ('127.0.0.1', 49670): hi
Echo from server: hi
Enter message (or 'quit' to stop): hello
Received from ('127.0.0.1', 49670): hello
Echo from server: hello
Enter message (or 'quit' to stop): how are you
Received from ('127.0.0.1', 49670): how are you
Echo from server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] closing connection.
[SERVER] client ('127.0.0.1', 49670) ended the connect
    -ion.
```

RESULT:

Thus, an echo-client server is implemented using TCP and UDP sockets.

EXP.NO:13
08/10/2026

PING PROGRAM

AIM:

To implement a user-defined ping program to check the connection and response time of a destination host over a network.

CODE:

```
import os
import platform
import subprocess
def ping(host="google.com", count=4)
    param = "-n" if platform.system().lower()
        == "windows" else "-c"
    command = ["ping", param, str(count),
        host]
    try:
        output = subprocess.check_output(
            command, universal_newlines=True)
        print(output)
    except Exception as e:
        print(f"Ping failed: {e}")
if __name__ == "__main__":
    ping("8.8.8.8")
```

OUTPUT:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes = 32 time = 6 ms TTL = 119
Reply from 8.8.8.8: bytes = 32 time = 9 ms TTL = 119
Reply from 8.8.8.8: bytes = 32 time = 5 ms TTL = 119
Reply from 8.8.8.8: bytes = 32 time = 9 ms TTL = 119
Ping statistics for 8.8.8.8: Sent = 4, Received = 4
```

Result:

Thus, it displays the destination host's IP address and response time.

EXP.NO: 14
08/10/2026

PACKET SNIFFING USING RAW SOCKETS

AIM:

To implement packet sniffing using raw sockets.

CODE:

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
        protocol_name = ""
        if protocol_name == 1:
            protocol_name = "ICMP"
        elif protocol_name == 6:
            protocol_name = "TCP"
        elif protocol_name == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown protocol"
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("-" * 50)
sniff(iface='Wi-Fi', prn=packet_callback, filter="ip"
      store=0)
```

OUTPUT:

Protocol: UDP
Source IP: 57.144.53.54
Destination IP: 172.16.57.165

Protocol: UDP
Source IP: 57.144.53.54
Destination IP: 172.16.57.165

Protocol: UDP
Source IP: 57.144.53.54
Destination IP: 172.16.57.165

Protocol: UDP
Source IP: 57.144.53.54
Destination IP: 172.16.57.165

Protocol: UDP
Source IP: 172.16.57.165
Destination IP: 57.144.53.54

RESULT:

Thus, the packet sniffing is successfully implemented using scapy and raw sockets to capture and display IP packet details.